

Open in app ↗

 MediumGet unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

Advanced Graphics Programming in Unreal, part 6

14 min read · Jun 25, 2025



William Mishra-Manning



Listen



Share



More

This is a seven-part article series.

Part 1: [Introduction](#)

Part 2: [Rendering Abstractions \(RHI, RDG\) and Threading](#)

Part 3: [Scenes, Views, and SceneViewExtension](#)

Part 4: [Global Shaders](#)

Part 5: [Simple Material Shaders](#)

Part 6: Advanced Material Shaders

Part 7: [Mesh-Material Shaders and Mesh Passes](#)

Advanced Material Shaders

In the last article we rebuilt the Game-of-Life effect using Material shaders instead of Global shaders, so that most of the effect behavior can be controlled by tech-artists in a special Material. However, the graph for this material is pretty hacky:

1. Post-process Materials only have 3 scalar outputs (Emissive RGB) which is extremely limiting.
2. We didn't provide any special inputs to the Material graph apart from the Sim

State texture.

3. We had to tack a strange switch statement onto the end of the graph to do different things for different passes.

In this article we'll add custom Material inputs and outputs, greatly expanding the flexibility and ease-of-use for artists playing with our custom effect.

Theory

Custom Material Inputs

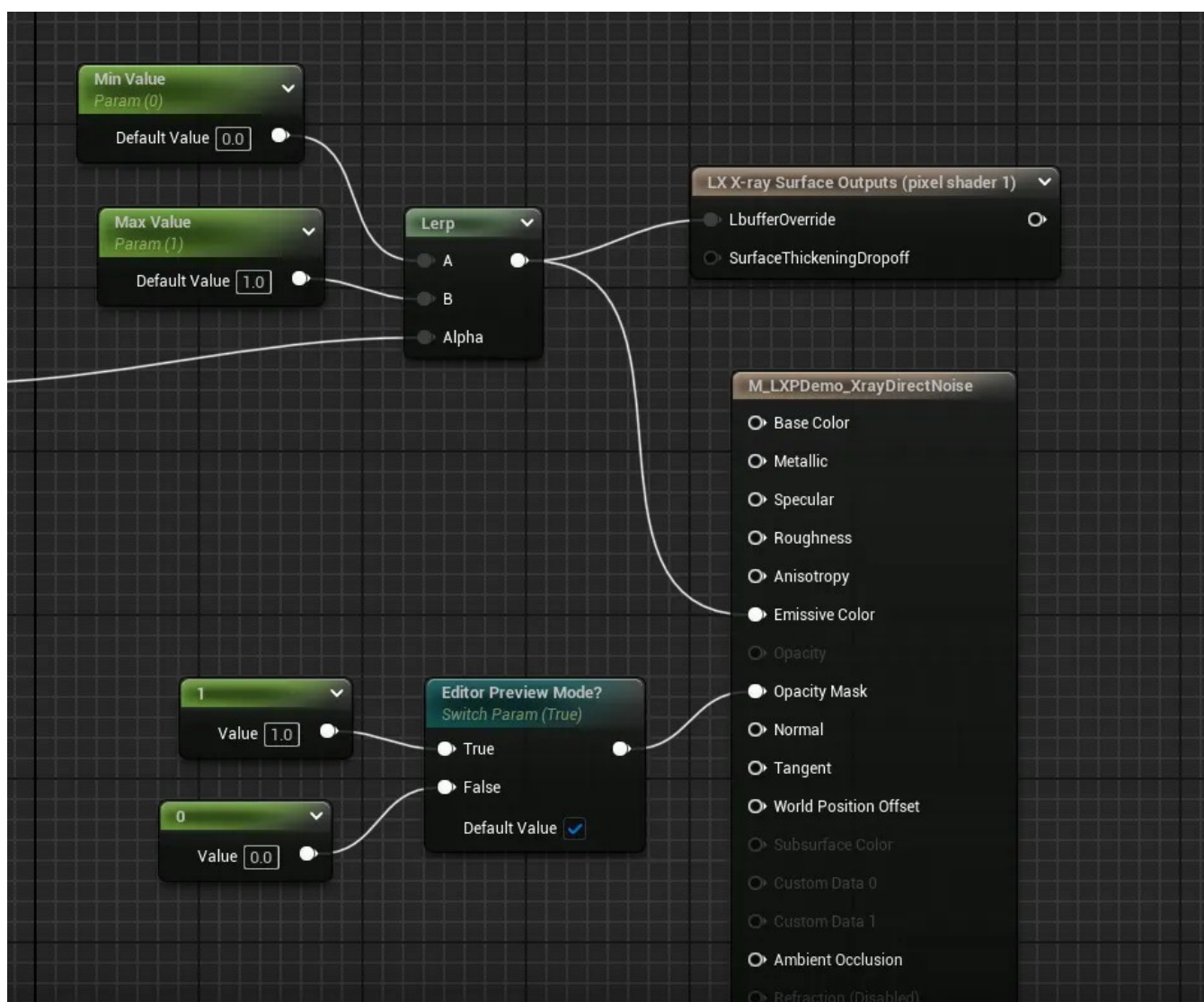
I alluded to custom inputs several times in the previous article. It's actually pretty easy to implement!

- Just before your shader includes Material code, add some `static global` variables for your inputs and a preprocessor token to hint that they exist.
 - The token is needed because each Material gets compiled against nearly *every* shader in its domain, including the one that renders the Material preview, so there will be shaders that use your graph without defining your globals.
- As soon as possible, especially before your shader executes any Material output pins (Get Emissive Color, Get Opacity, etc), make sure to set those globals.
- Expose them to the Material graph using a Material Function asset, containing a custom HLSL node that reads and returns their values. This node uses the above preprocessor token, returning stub values if running within a different shader.

Custom Material Outputs

Custom outputs are defined as a new `UCLASS()` inheriting from `UMaterialExpressionCustomOutput`. They, along with Material Function assets, are the **only** extension you can make to Material graphs without modifying the engine. If you try inheriting from other node classes you'll get linker errors.

A custom output node looks exactly like the standard Material outputs with a beige title bar. There are several built-in examples you may have seen already, such as Strata and Landscape.



A custom output from our Xray renderer at work

Each new output node can apply to a *single* shader frequency, so you usually want about one per Material shader. Do the following to configure it assuming you have a hard-coded set of simple output pins (for more complex cases look at the base class and all the examples in-engine):

For convenience, give it the `UCLASS` properties `CollapseCategories,`
`HideCategories=Object`.

Also give it a `DisplayName`, which appears in the popup menu when selecting a Material node.

Define each of your output pins as fields:

```
UPROPERTY(meta=(RequiredInput=true))  
FExpressionInput MyPin;
```

The name of the graph pin comes from the name of the field. If `RequiredInput` is true, users are forced to write to this pin whenever the node exists.

If you want the pin to provide a fallback textbox, for the user to enter a constant value, add a companion field:

```
UPROPERTY(meta=(OverridingInputProperty=MyPin))  
float MyPinF;
```

Define any other `UPROPERTY` fields you want. Properties marked with `EditAnywhere` will appear in the node's Details panel.

Within an `#if WITH_EDITOR` block, override the following functions from the `UMaterialExpression` interface:

- `void GetCaption(TArray<FString>& output) const` : generates the title of this node as it should appear in the graph.
- `int32 GetNumOutputs() const` : tells the shader compiler how many outputs pins you have.
- `EShaderFrequency GetShaderFrequency()` : tells the shader compiler which frequency these outputs are used in (compute, pixel, vertex, etc).
- `int32 Compile(FMaterialCompiler* compiler, int32 pinIdx)` : generates code for the given output pin. This function is covered in detail below.

Override the following functions (*not* within an `#if WITH_EDITOR` block) from the `UMaterialExpressionCustomOutput` interface:

- `FString GetFunctionName() const` : a prefix that is used in the shader to query your outputs; it must be a legal HLSL token and probably shouldn't start with an underscore to be safe. This prefix is covered in detail below.
- `FString GetDisplayName() const` : I can't figure out where this name actually appears, but it doesn't need to be a valid HLSL token. You can just return the same value as in `GetCaption()` or `DisplayName`.

The `Compile` function is very important, but not too complicated. Generally you check whether the pin is set, compile it if so, and otherwise fall back to a constant value or omit the pin entirely. Here is a sample from our own X-ray renderer, which offers two output pins:

```
int32 codeID;
if (pinIdx == 0)
{
    codeID = LbufferOverridePin.IsConnected() ?
        LbufferOverridePin.Compile(compiler) :
        INDEX_NONE;
}
else if (pinIdx == 1)
{
    codeID = SurfaceThickeningDropoffPin.IsConnected() ?
        SurfaceThickeningDropoffPin.Compile(compiler) :
        compiler->Constant(SurfaceThickeningDropoffConst);
}
else //Unknown pin idx
{
    codeID = INDEX_NONE;
}
return compiler->CustomOutput(this, pinIdx, codeID);
```

NOTE: even if the pin isn't set to anything, we still want to call `compiler->CustomOutput(...)`, passing it `INDEX_NONE` for the output value. This generates shader code

to indicate the pin is empty.

Once the above is in place, the compiler will start generating new Material code in *all* shaders. New preprocessor tokens tell whether each of our output pins are written to, and new functions tell us how to compute them. This is where `GetFunctionName()` comes in; for example if it returns `TEXT("MyCustomOuts")` then you will have access to the following:

- One preprocessor token per output pin, telling you whether that pin has a value
 - `HAVE_MyCustomOuts0` , `HAVE_MyCustomOuts1` , etc.
- A function you can call to execute the pin, IF it has a value —
 - `MyCustomOuts0(matParameters)` , `MyCustomOuts1(matParameters)` , etc.

Here is some shader code from our X-ray render effect:

```
//Did the Material provide an override for the L-buffer value?
#if HAVE_GetXrayPixelOutput0
    outLbuffer = GetXrayPixelOutput0(materialParameters);
//If not, then compute it the standard way.
#else
    outLbuffer = length(input.WorldPosition.xyz) * (isFrontFace ? -1 : 1);
#endif
```

Demo

Let's start with custom inputs.

Custom Inputs

Of all our shaders only *Simulate* really has extra data we could give to the Material Graph, namely the current cell state and the neighboring cell states.



All our Material shaders already define a preprocessor token to indicate which one is being compiled. Let's add static globals alongside them in *Simulate*:

```
//At the top of Simulate.usf:

#include "/EGP/ScreenPass/pre.ush"

#define GOL_PASS_SIMULATE 1
//Custom Material inputs!
static float MaterialDeltaSeconds;
static float2 MaterialNeighborStates[9];

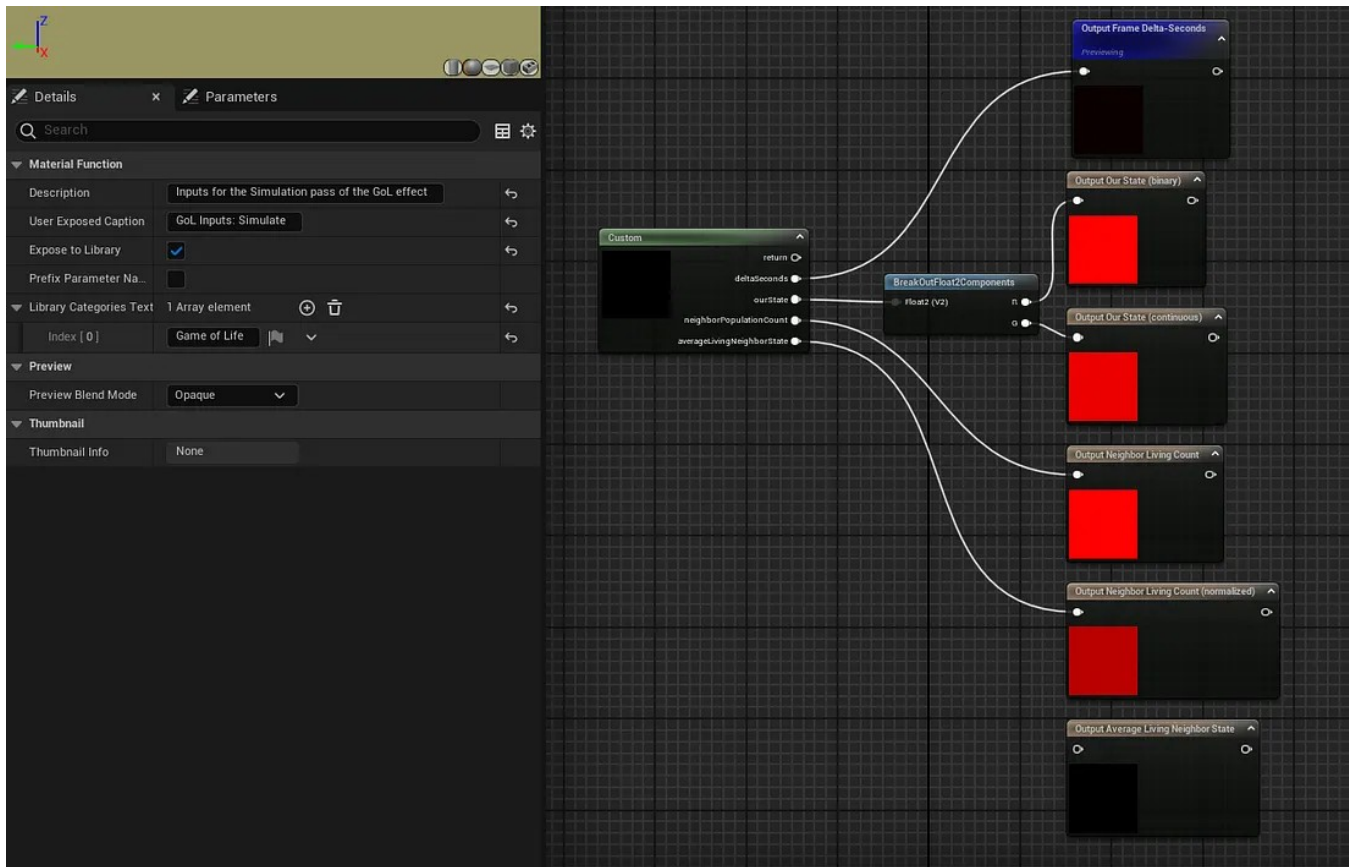
#include "/EGP/ScreenPass/post.ush"

...

//In the main function, just after setting up 'matInputs' and 'matParams',
// and after sampling our neighborhood of 9 cells 'prevStates':
MaterialDeltaSeconds = DeltaSeconds;
for (int i = 0; i < 9; ++i)
    MaterialNeighborStates[i] = prevStates[i];
//Now when we invoke Material output pins, they can read those globals.
```

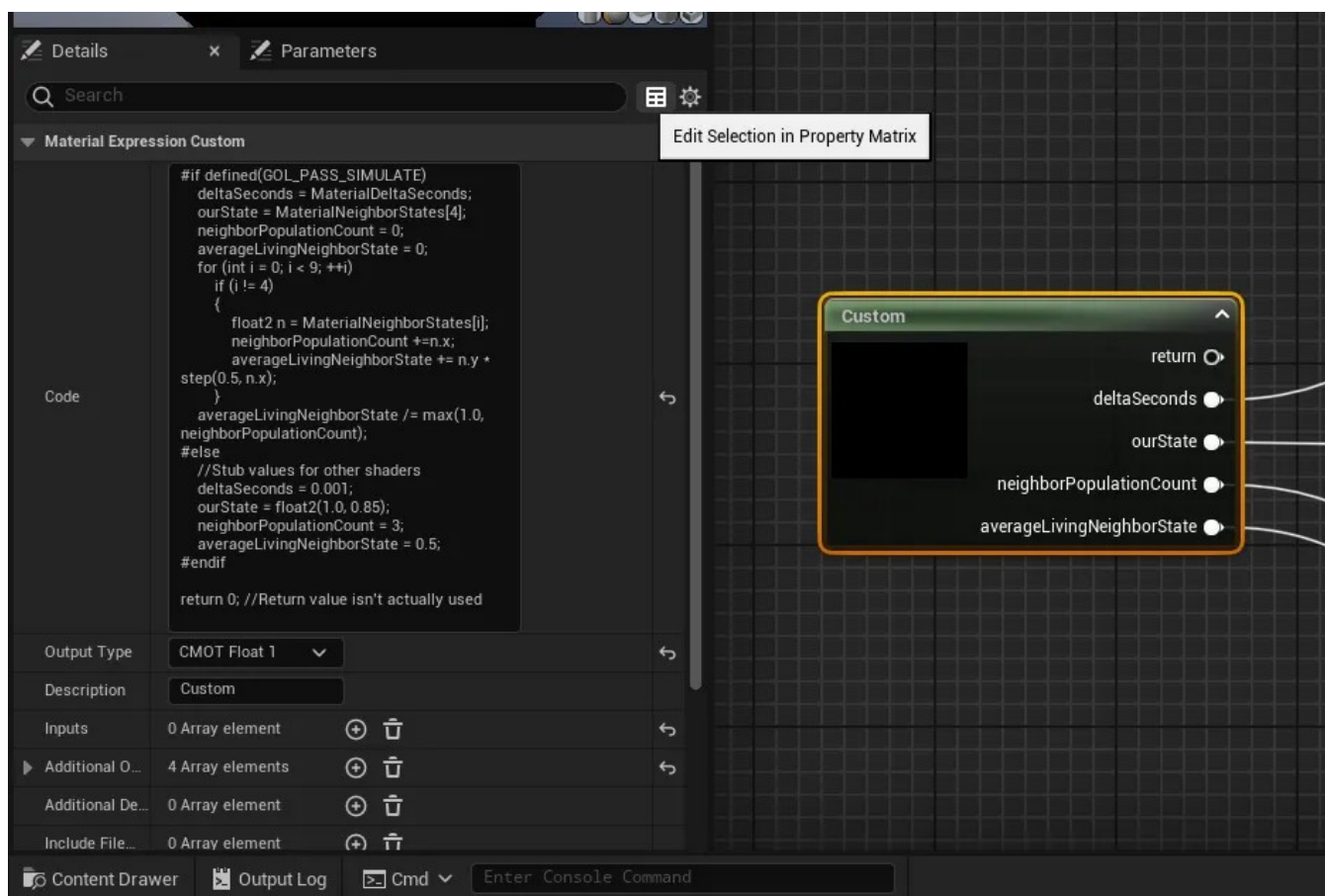
Next add a Material Function asset which reads those globals. Since the shader and effects code is all in a plugin, *GOL_Demo/*, this asset will go in the plugin content

folder `GOL_Demo/Content`. Plugin content folders aren't visible in the editor by default, but they always exist.



Pay attention to the asset settings on the left too!

(I just noticed the above screenshot screws up the last two outputs, but from their names you should be able to infer what I meant)



The Custom HLSL code that reads our inputs. Also provided below.

```
#if defined(GOL_PASS_SIMULATE)
    deltaSeconds = MaterialDeltaSeconds;
    ourState = MaterialNeighborStates[4];

    neighborPopulationCount = 0;
    averageLivingNeighborState = 0;
    for (int i = 0; i < 9; ++i)
        if (i != 4)
        {
            float2 n = MaterialNeighborStates[i];
            neighborPopulationCount += n.x;
            averageLivingNeighborState += n.y * step(0.5, n.x);
        }
    averageLivingNeighborState /= max(1.0, neighborPopulationCount);

#else
    //Stub values for other shaders
```

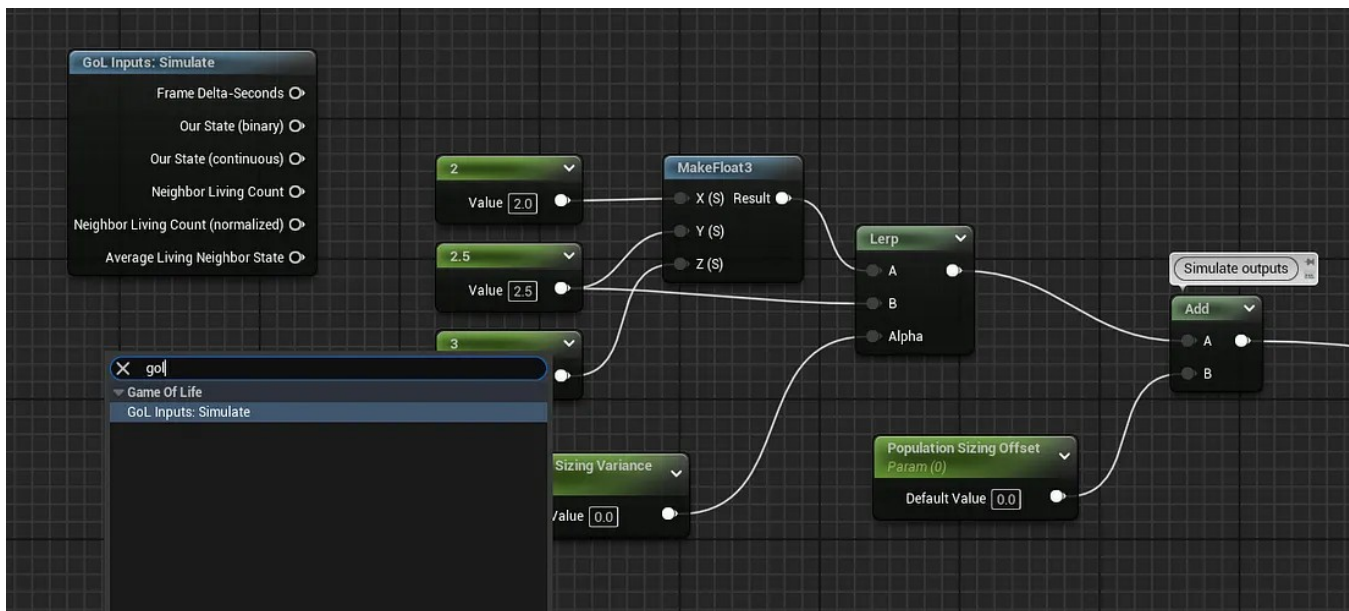
```

deltaSeconds = 0.001;
ourState = float2(1.0, 0.85);
neighborPopulationCount = 3;
averageLivingNeighborState = 0.5;
#endif

return 0; //value not actually used

```

Now you can drop this input into your graph and use it!



With 'Expose to Library' checked, the MaterialFunction shows up in the context menu

Custom Outputs: Init shader

Outputs are more complicated than inputs. To reiterate, we are replacing that hacky switch statement into Emissive Color, with totally separate output nodes per shader.

Start with the simplest case, *Init*. We need one required output for the discrete state, and one optional output for the continuous state which defaults to the discrete value.

```

//In a C++ header:

#include "Materials/MaterialExpressionCustomOutput.h"

UCLASS(CollapseCategories, HideCategories=Object,

```



```

        DisplayName="GoL Outputs: Init")
class GOL_DEMO_API UMaterialExpressionGoLInitOutputs
: public UMaterialExpressionCustomOutput
{
    GENERATED_BODY()
public:

    //Automatically snapped to 0 or 1.
    UPROPERTY(meta=(RequiredInput=true))
    FExpressionInput InitialBinaryState;

    //If not set, defaults to the binary state value
    //    (before it's snapped to 0 or 1).
    UPROPERTY(meta=(RequiredInput=false))
    FExpressionInput InitialContinuousState;

    virtual FString GetFunctionName() const override {
        return TEXT("GoL_Outputs_Init_");
    }
    virtual FString GetDisplayName() const override {
        return TEXT("GoL Outputs: Init");
    }

    #if WITH_EDITOR
        virtual void GetCaption(TArray<FString>& output) const override {
            output.Add(TEXT("Game of Life Outputs: Init pass"));
        }
        virtual int32 GetNumOutputs() const override { return 2; }
        virtual EShaderFrequency GetShaderFrequency() override {
            return SF_Pixel;
        }
        virtual int32 Compile(class FMaterialCompiler*,
                            int32 pinIdx) override;
    #endif
};

...

//In a C++ cpp file:

#include "MaterialCompiler.h"

#if WITH_EDITOR
int32 UMaterialExpressionGoLInitOutputs::Compile(FMaterialCompiler* compiler,
                                                int32 pinIdx)
{

```

```
int32 codeID;
if (pinIdx == 0)
{
    codeID = InitialBinaryState.IsConnected() ?
        InitialBinaryState.Compile(compiler) :
        INDEX_NONE;
}
else if (pinIdx == 1)
{
    //The default behavior is controlled in the shader, not here
    codeID = InitialContinuousState.IsConnected() ?
        InitialContinuousState.Compile(compiler) :
        INDEX_NONE;
}
else
{
    codeID = INDEX_NONE;
}
return compiler->CustomOutput(this, pinIdx, codeID);
}
#endif

...

//In Init.usf, replace the code that reads from Material Emissive pin
//    with this:

//Read the starting state from the Material.

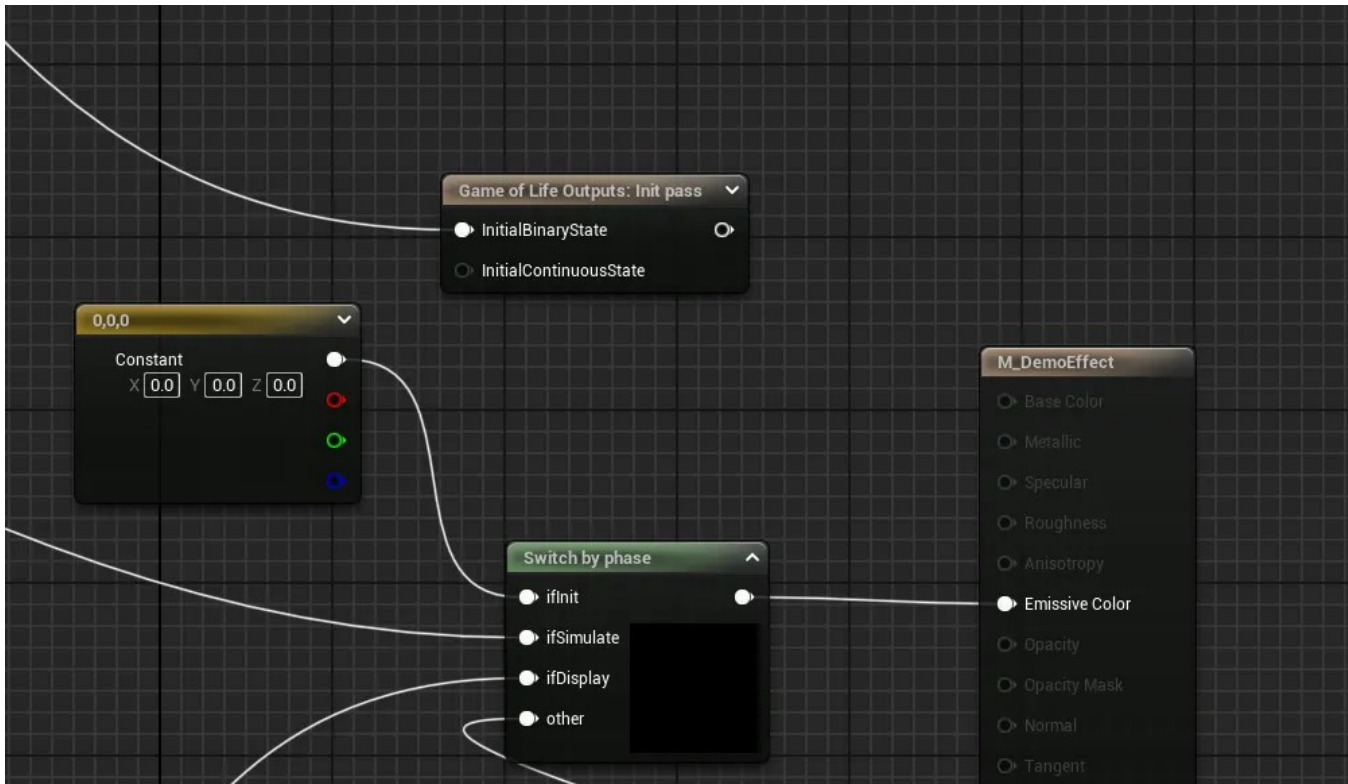
// 1. The "discrete state" output.
#if HAVE_GoL_Outputs_Init_0
    newState.x = GoL_Outputs_Init_0(matParams);
#else
    //The first pin must be set,
    //    but unrelated post-process Materials will compile without it,
    //    so this branch happens sometimes and we'll use a dummy value.
    newState.x = 0;
#endif

// 2. The "continuous state" output.
#if HAVE_GoL_Outputs_Init_1
    newState.y = HAVE_GoL_Outputs_Init_1(matParams);
#else
    //If not set, the pin defaults to the other one's value.
    newState.y = newState.x;
#endif
```



```
//Sanitize the inputs from the Material.  
newState.x = step(0.5, SafeSaturate(newState.x));  
newState.y = SafeSaturate(newState.y);
```

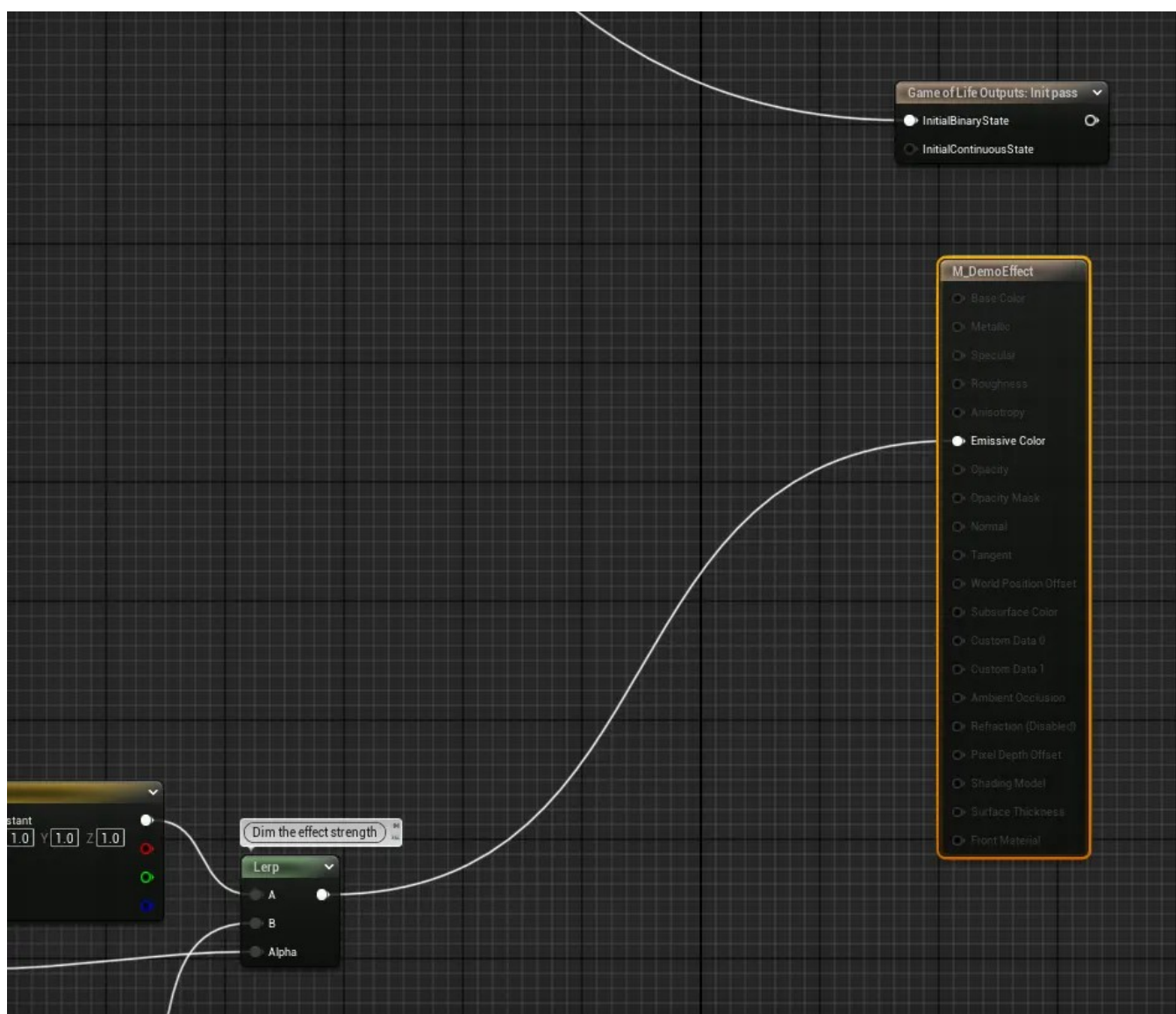
Finally, go into the Material and use these new output pins instead of the switch statement.



Run the demo and see that the effect is unchanged.

Custom Outputs: Display shader

It's actually quite natural for Display to output to the Emissive Color pin, since this shader does exactly the same thing as a real post-process Material! So let's plug it directly into Emissive Color.



Delete the switch node entirely; only Display writes to Emissive Color now.

This does mean we have less control over our Material preview. If your project ends up with many different effect Materials, it can be nice to clearly indicate which is which by writing a specific color in the Material Preview, so you may want to bring that switch node back.

More importantly, we broke the Simulate pass. Let's fix that and finish this article.

Multi-stage Inputs and Outputs: Simulate shader

What's interesting about the Simulate pass is that it involves a *sequence* of decisions: read the neighboring cells, decide on the discrete state, compute a speed for the continuous state, then apply that speed towards the discrete state. To give maximum

control to the Material graph, we will split the inputs and outputs for this shader into several stages.

1. Read neighboring cells and decide whether ours is living or dead.
 - Inputs:** cell states (already provided in the Material Function we created earlier)
 - Outputs:** several Game of Life thresholds — underpopulation, ideal population, and overpopulation.
2. Given our new alive/dead discrete state, decide on the new continuous state.
 - Inputs:** the previous inputs, plus the new discrete state for our cell, and the associated `severityT` value computed in the shader.
 - Outputs:** the new continuous state for our cell

This setup means we need more inputs coming in from a second Material Function, for our second stage. We also need two different output nodes for each stage's output.

In a C++ header, add the two new output nodes:

```
UCLASS(CollapseCategories, HideCategories=Object,
       DisplayName="GoL Outputs: Simulate (pt 1)")
class GOL_DEMO_API UMaterialExpressionGoLSimulate10Outputs
    : public UMaterialExpressionCustomOutput
{
    GENERATED_BODY()
public:

    UPROPERTY(meta=(RequiredInput=false),
              DisplayName="Threshold: Underpopulated")
    FExpressionInput ThresholdTooFew;
    UPROPERTY(meta=(OverridingInputProperty=ThresholdTooFew))
    float ThresholdTooFewConst = 2.0f;

    UPROPERTY(meta=(RequiredInput=false),
              DisplayName="Threshold: Ideal")
    FExpressionInput ThresholdResurrect;
    UPROPERTY(meta=(OverridingInputProperty=ThresholdResurrect))
    float ThresholdResurrectConst = 2.5f;
```

```

UPROPERTY(meta=(RequiredInput=false),
           DisplayName="Threshold: Overpopulation")
FExpressionInput ThresholdTooMany;
UPROPERTY(meta=(OverridingInputProperty=ThresholdTooMany))
float ThresholdTooManyConst = 3.0f;

virtual FString GetFunctionName() const override {
    return TEXT("GoL_Outputs_Simulate_Pt1_");\
}
virtual FString GetDisplayName() const override {
    return TEXT("GoL Outputs: Simulate (part 1)");
}

#if WITH_EDITOR
    virtual void GetCaption(TArray<FString>& output) const override {
        output.Add(TEXT("Game of Life Outputs: “
                        “Simulate pass (part 1)"));
    }
    virtual int32 GetNumOutputs() const override { return 3; }
    virtual EShaderFrequency GetShaderFrequency() override {
        return SF_Compute;
    }
    virtual int32 Compile(FMaterialCompiler*, int32 pinIdx) override;
#endif
};

UCLASS(CollapseCategories, HideCategories=Object,
       DisplayName="GoL Outputs: Simulate (pt 2)")
class GOL_DEMO_API UMaterialExpressionGoLSimulate2Outputs
    : public UMaterialExpressionCustomOutput
{
    GENERATED_BODY()
public:

    //If not provided, the continuous state will match the discrete one.
    UPROPERTY(meta=(RequiredInput=false))
    FExpressionInput ContinuousValue;

    virtual FString GetFunctionName() const override {
        return TEXT("GoL_Outputs_Simulate_Pt2_"); }
    virtual FString GetDisplayName() const override {
        return TEXT("GoL Outputs: Simulate (part 2)");
    }

    #if WITH_EDITOR
        virtual void GetCaption(TArray<FString>& output) const override {
            output.Add(TEXT("Game of Life Outputs: “
                            “Simulate pass (part 2)"));
        }
    #endif
};

```

```

    }
    virtual int32 GetNumOutputs() const override { return 1; }
    virtual EShaderFrequency GetShaderFrequency() override {
        return SF_Compute;
    }
    virtual int32 Compile(FMaterialCompiler*, int32 pinIdx) override;
#endif
};

```

Implement them in a cpp file:

```

if WITH_EDITOR
int32 UMaterialExpressionGoLSimulate1Outputs::Compile(FMaterialCompiler* compiler,
                                                    int32 pinIdx)
{
    int32 codeID;
    auto doPin = [&](int32 i, FExpressionInput& pin, float* fallback)
    {
        if (pinIdx != i)
            return false;

        if (pin.IsConnected())
            codeID = pin.Compile(compiler);
        else if (fallback)
            codeID = compiler->Constant(*fallback);
        else
            codeID = INDEX_NONE;
        return true;
    };

    if (!doPin(0, ThresholdTooFew, &ThresholdTooFewConst) &&
        !doPin(1, ThresholdResurrect, &ThresholdResurrectConst) &&
        !doPin(2, ThresholdTooMany, &ThresholdTooManyConst))
    {
        codeID = INDEX_NONE;
    }
    return compiler->CustomOutput(this, pinIdx, codeID);
}
int32 UMaterialExpressionGoLSimulate2Outputs::Compile(FMaterialCompiler* compiler,
                                                    int32 pinIdx)
{
    int32 codeID;
    auto doPin = [&](int32 i, FExpressionInput& pin, float* fallback)
    {

```

```

        if (pinIdx != i)
            return false;

        if (pin.IsConnected())
            codeID = pin.Compile(compiler);
        else if (fallback)
            codeID = compiler->Constant(*fallback);
        else
            codeID = INDEX_NONE;
        return true;
    };

    if (!doPin(0, ContinuousValue, nullptr))
    {
        codeID = INDEX_NONE;
    }
    return compiler->CustomOutput(this, pinIdx, codeID);
}
#endif

```

And finally, update *Simulate.usf* to invoke the Material in stages.

```

//Near the top, where we define Material inputs:

//Stage 1
static float MaterialDeltaSeconds;
static float2 MaterialNeighborStates[9];

//Stage 2
static float MaterialNewDiscreteValue;
static float MaterialDiscreteSeverityT;

...

//At the point where we read the Material Emissive pin,
//    replace it and all following code with this.

//Run stage 1 of the Material graph's logic.
MaterialDeltaSeconds = DeltaSeconds;
for (int i = 0; i < 9; ++i)
    MaterialNeighborStates[i] = prevStates[i];
float thresholdTooFew =

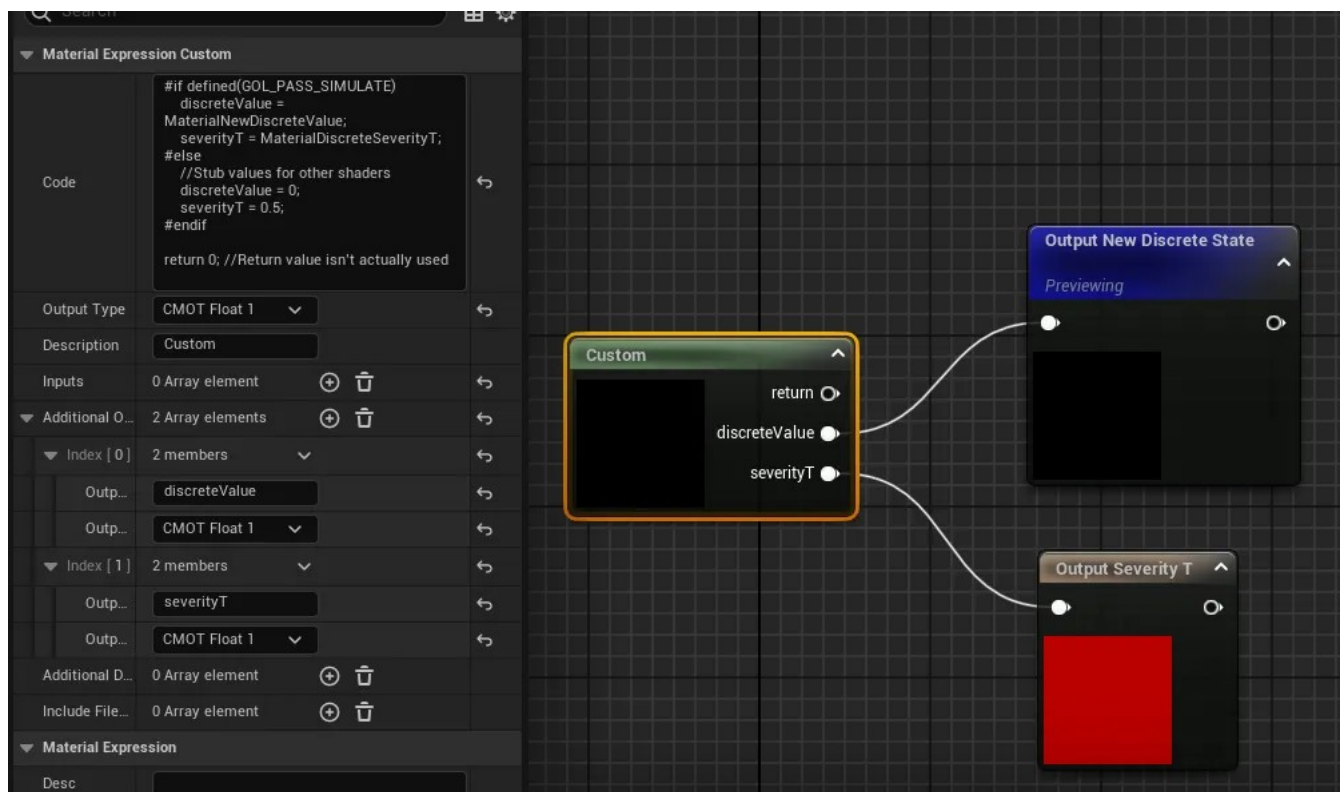
```

```
#if HAVE_GoL_Outputs_Simulate_Pt1_0
    GoL_Outputs_Simulate_Pt1_0(matParams)
#else
    2.0
#endif
;
float thresholdResurrect =
    #if HAVE_GoL_Outputs_Simulate_Pt1_1
        GoL_Outputs_Simulate_Pt1_1(matParams)
    #else
        2.5
    #endif
;
float thresholdTooMany =
    #if HAVE_GoL_Outputs_Simulate_Pt1_2
        GoL_Outputs_Simulate_Pt1_2(matParams)
    #else
        3
    #endif
;
//Count the number of living neighbors surrounding us.
float neighborStrength = 0;
const float maxStrength = 8.0;
for (int i = 0; i < 9; ++i)
    if (i != ourIdx)
        neighborStrength += prevStates[i].x;
//Apply the usual Game of Life rules to our cell,
//    dying/living/resurrecting based on living neighbor count.
//Also calculate a "severity" estimating the strength of the pull towards that
float targetValue, severityT;
if (neighborStrength < thresholdTooFew)
{
    //Die off, from underpopulation.
    targetValue = 0;
    severityT = 1.0 - (neighborStrength / thresholdTooFew);
}
else if (neighborStrength < thresholdResurrect)
{
    //Maintain current living status, from a normal population.
    targetValue = prevStates[ourIdx].x;
    severityT = 1.0;
}
else if (neighborStrength <= thresholdTooMany)
{
    //Come alive, from an ideal population.
    targetValue = 1;
    severityT = (neighborStrength - thresholdResurrect) /
        max(0.0001, thresholdTooMany - thresholdResurrect);
}
```



```
}  
else  
{  
    //Die, from overpopulation.  
    targetValue = 0;  
    severityT = (neighborStrength - thresholdTooMany) /  
                max(0.0001, maxStrength - thresholdTooMany);  
}  
  
//Run stage 2 of the Material graph's logic to pick a continuous value.  
MaterialNewDiscreteValue = targetValue;  
MaterialDiscreteSeverityT = severityT;  
float smoothValue =  
    #if HAVE_GoL_Outputs_Simulate_Pt2_0  
        saturate(GoL_Outputs_Simulate_Pt2_0(matParams))  
    #else  
        targetValue  
    #endif  
;  
  
//Output the result.  
NextSimStateTex[pixel] = float2(targetValue, smoothValue);
```

All that's left is to update the assets. Rename the existing simulation input Material Function to be “part 1”, and add a corresponding “part 2”:



Code available below

```
#if defined(GOL_PASS_SIMULATE)
    discreteValue = MaterialNewDiscreteValue;
    severityT = MaterialDiscreteSeverityT;

#else
    //Stub values for other shaders
    discreteValue = 0;
    severityT = 0.5;
#endif
return 0; //Return value isn't actually used
```

Finally, go back to the Material and use these custom outputs. Below is a graph that reproduces the original behavior.

The new old simulation pass (click for full resolution)

Playing with the Material

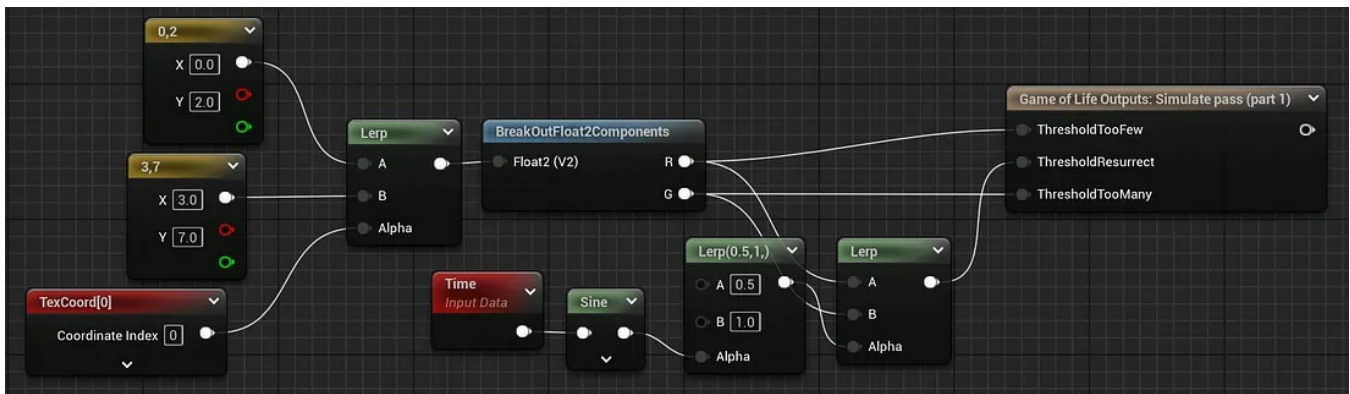
You can get very nice visuals by lowering the speed when animating towards a dead state, leaving ghostly trails on the screen.

(click for full resolution)

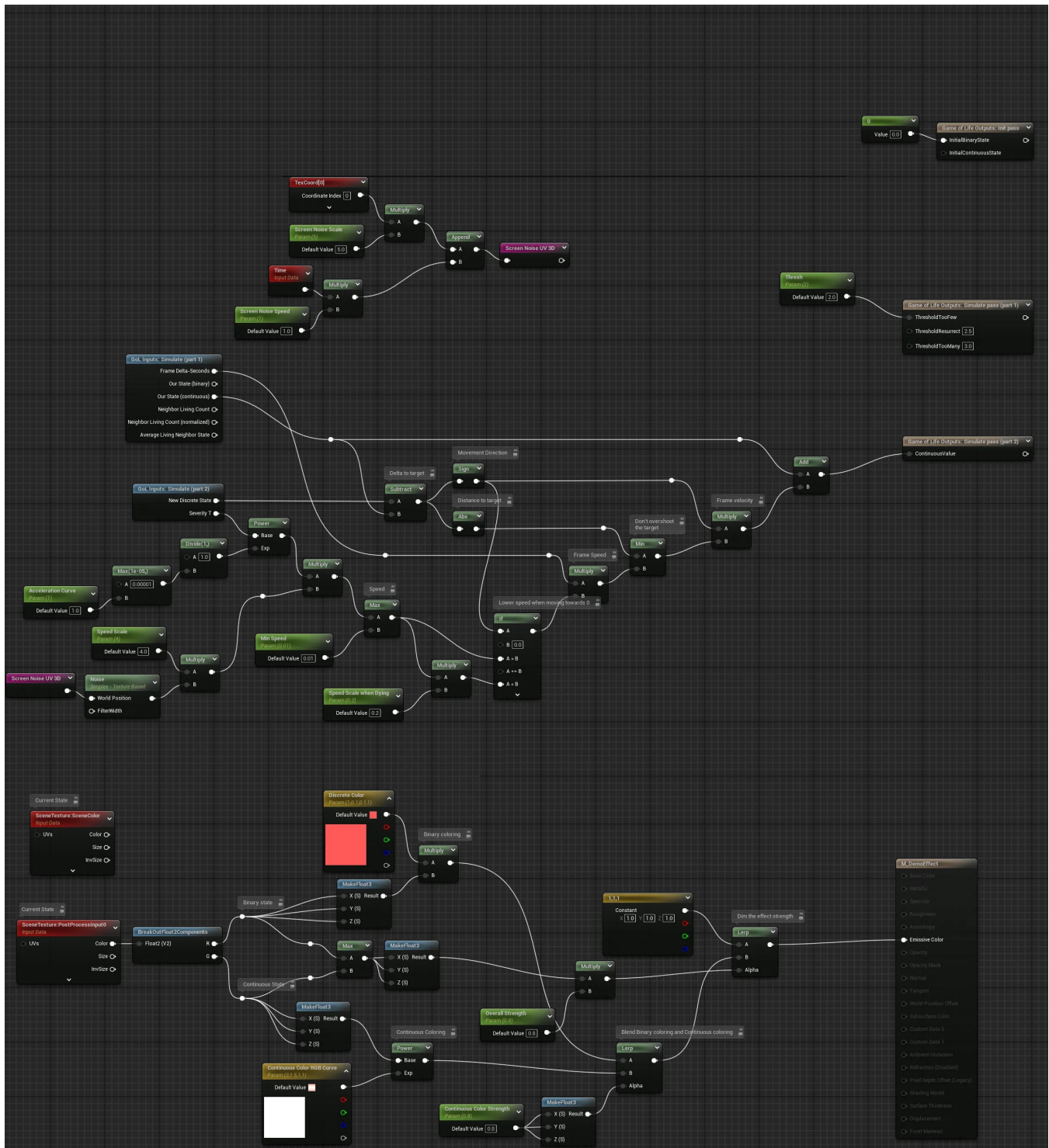
Let's add screen-space animated noise to one of the parameters to get a little more variety in the ghost trails.

(click for full resolution)

If you want to see the effect of using different thresholds, here's a graph that varies the Underpopulation and Overpopulation thresholds with screen UV, and the Resurrection threshold in between them over time. The released demo uses this graph.



From here, the sky's the limit! If your focus is on screen-space and compute shaders then you can consider this the end of the series. However the next article will show how to make a 3D scene pass, drawing actors into our simulation.



An overview of the final Material graph, with each pass neatly organized

Code

The Material shaders are upgraded in the *part/6* branch:

[heyx3/UnrealExtendedGraphicsProgrammingDemo at part/6](#)

Details

I can think of several improvements to increase the artists' control over the Game of Life effect. I'll leave them as exercises for the reader:

- Offer extra sim state textures that Materials can use however they want throughout each pass. For example, the artist could decide to use one channel to store each cell's time since it was last alive. Add fields to the pass object to let artists configure the existence/format of those extra textures.
- Provide more high-level settings in the custom pass object, such as the blend mode used for the Display shader.
- Provide a parameter in the custom pass object to change the destination of the Display shader. Write to G-buffer targets like Base Color or Metallic instead of the final color!
- Provide an option to compute the state texture's mips before Display and/or Simulate, so that the artist can do more interesting things with clusters of cells.

Custom output nodes seem capable of all sorts of tricks. For example you could have a dynamic number of output pins based on some `UPROPERTY` data. You can also apparently override `CompilePreview()` to compile your outputs differently for the Material preview vs the game world. The class

`UMaterialExpressionPhysicalMaterialOutput` looks like a good example for more complex behaviors.

My distortion effect in *Leave Them Behind* actually has six shader stages: two simpler shaders, a final simulation shader with three stages, then the display shader. The HLSL and Material Functions for that third shader look heinous, but the Material graph is still easy to read because the steps happen in a clear order.



The simulation shaders for DisortionFX

Material code does not cache the results of custom output pins, which means you can invoke the same pin more than once while changing your input globals to get

different results! This allows you to define loops in your effect code, where your shader controls the outside and the Material controls the inside. I can think of at least one creative use for this feedback loop, which will be included in the next/final version of the demo project as a bonus effect. This trick does *not* work with built-in output pins like Emissive Color.

Unreal Engine 5

Graphics Programming

Shaders

Technical Art



Edit profile

Written by William Mishra-Manning

59 followers · 5 following

Graphics Programming, Procedural Generation

Responses (2)



William Mishra-Manning

What are your thoughts?



Scott

Jun 26, 2025



This series is absolutely brilliant, and the amount of effort you've invested in researching and carefully documenting so many internal behaviors and quirks is evident in every article. Every hour you have invested

in this series becomes an hour... [more](#)



7



1 reply

[Reply](#)



Ardentis Ignis

Mar 17



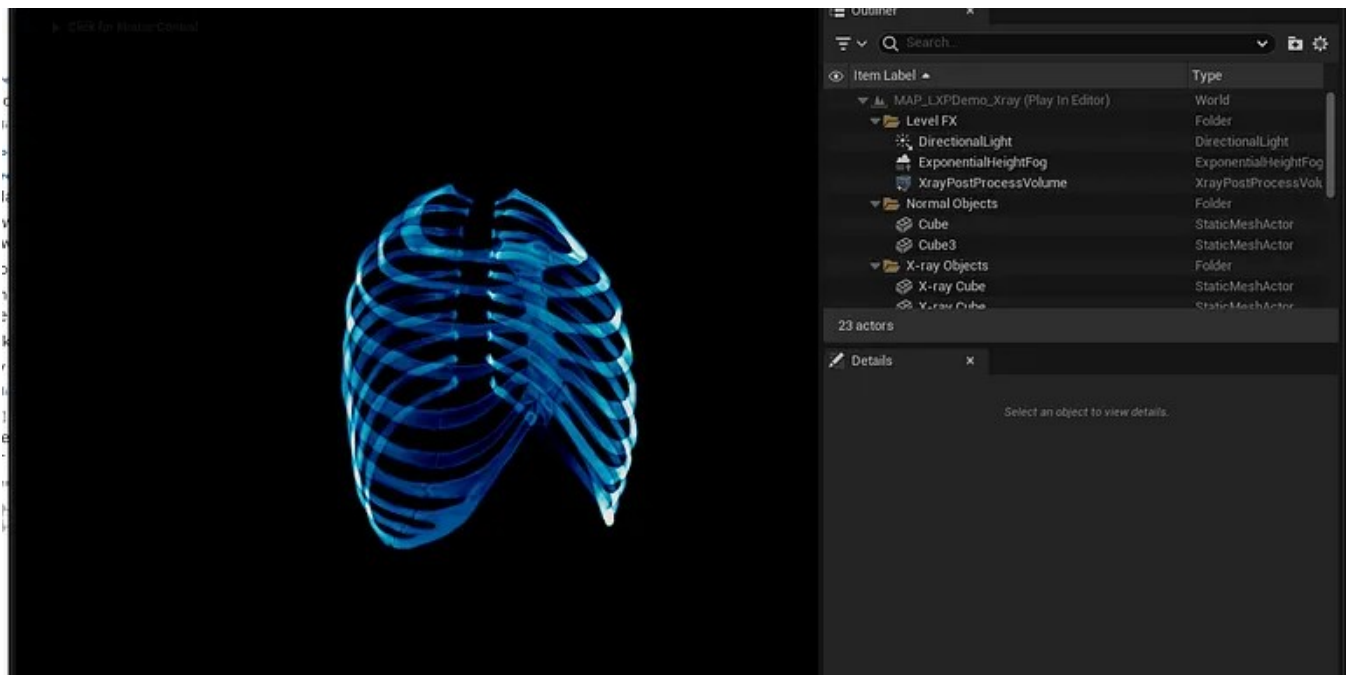
This is a treasure, and I thank you for all the effort and love you poured into this amazing series.



5

[Reply](#)

More from William Mishra-Manning



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 1

This is a seven-part article series.

Jun 24, 2025

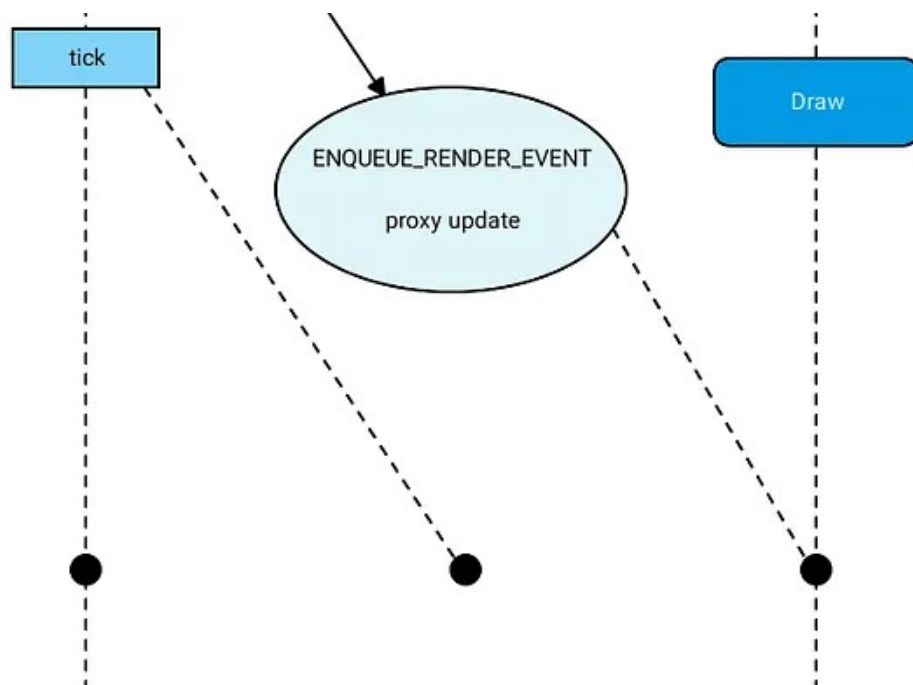


31



1





William Mishra-Manning

Advanced Graphics Programming in Unreal, part 2

This is a seven-part article series.

Jun 24, 2025



16



1





William Mishra-Manning

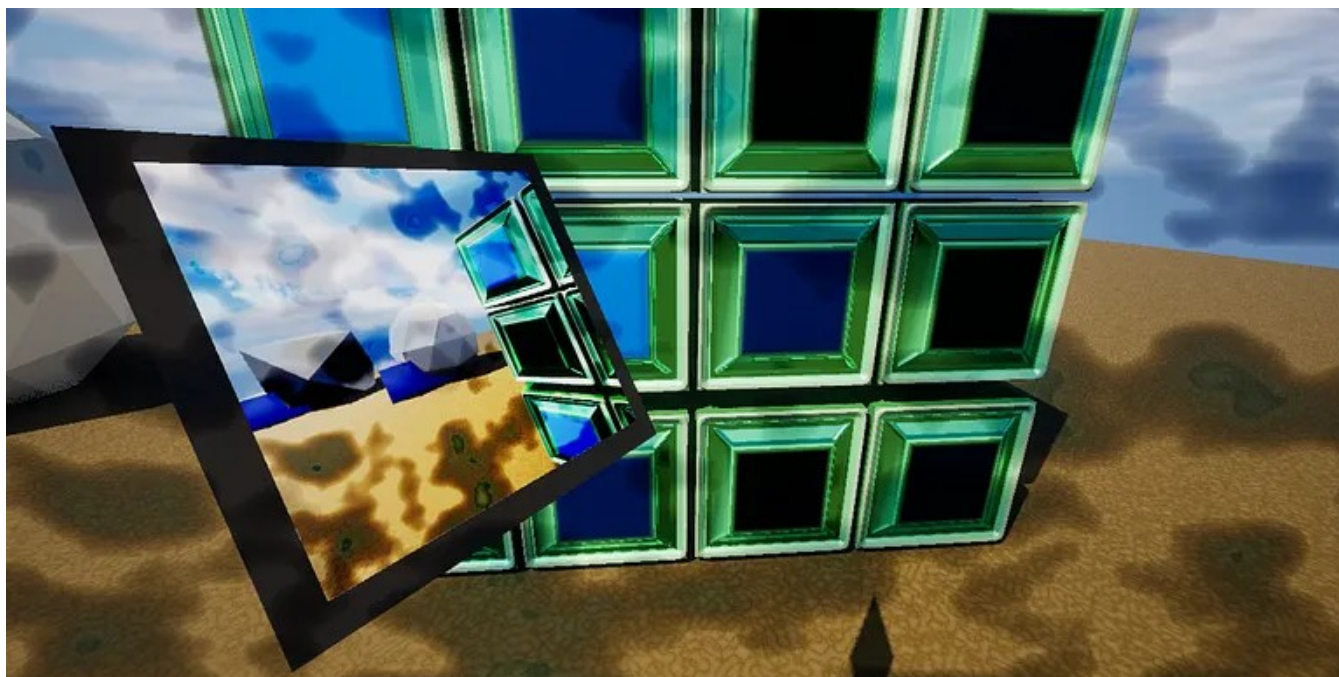
Advanced Graphics Programming in Unreal, part 3

This is a seven-part article series.

Jun 24, 2025



6



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 4

This is a seven-part article series.

Jun 25, 2025



18



See all from William Mishra-Manning

Recommended from Medium



James Roha

Gaussian splatting: a complete student guide to 3D capture in 2026

3D Gaussian Splatting has transformed how we capture and render real-world scenes, enabling photorealistic results at 100+ frames per...

Feb 13 🖱️ 7



 AmirHossein Aghajari

Liquid Glass: iOS Effect Explanation

Apple's Liquid Glass effect with shaders: math, distortion & chromatic aberration

Nov 23, 2025  240  1



 Nikhil Maurya

RDG in Practice: From Shader Parameters to a Working Pass

The Render Dependency Graph (RDG) is Unreal Engine's way of describing GPU work : passes, resources and lifetimes. So the renderer can...

Jan 15  4



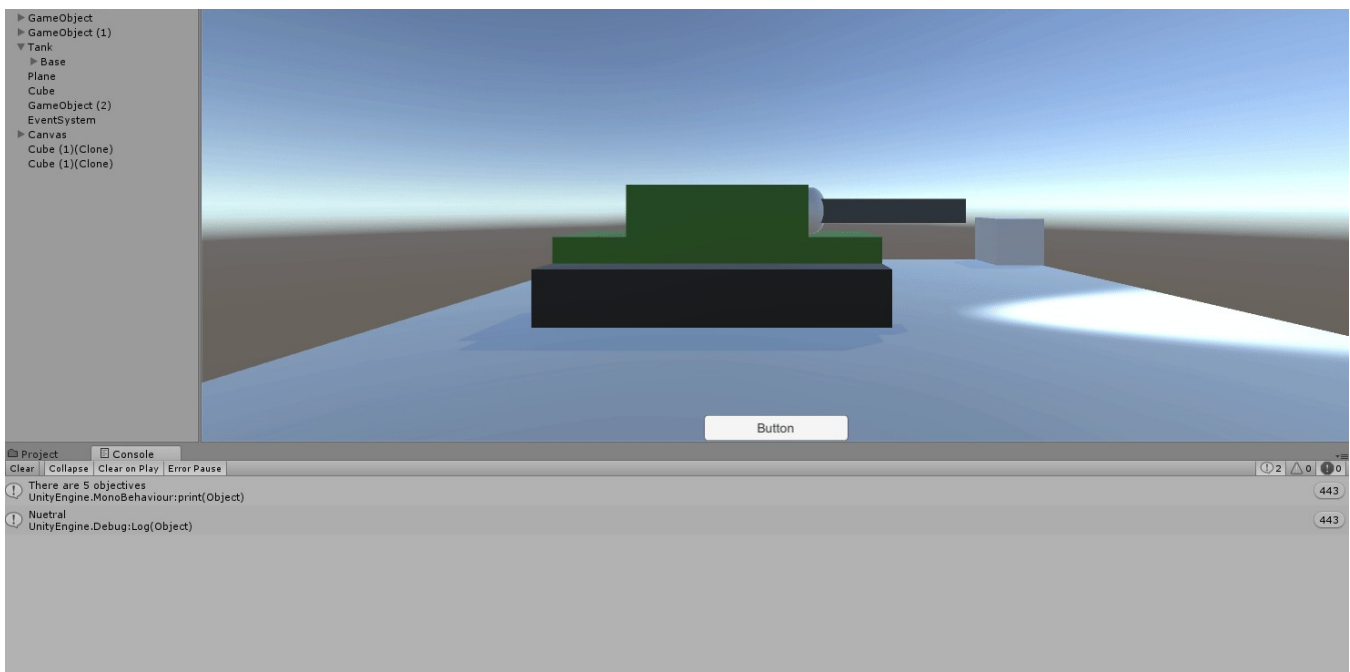
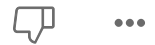


a Akash Kadam

Setting Up FreeRTOS on STM32 / ESP32 from Zero A Hands-On Beginner's Guide

If you've already understood what FreeRTOS is and why bare-metal eventually breaks, the next natural question is:

Jan 29 🖱️ 1



 Zac Bogner

Switching from Unity to Unreal

In the previous article, I wrote about Switch Statements in Unity.

Dec 22, 2025  3



1. Start with Bayes' Rule:

$$P(x|z) = \frac{P(z|x)P(x)}{P(z)}$$

2. Divide by the "Free" state ($\neg x$) to create an Odds Ratio:

$$\frac{P(x|z)}{P(\neg x|z)} = \frac{P(z|x)}{P(z|\neg x)} \cdot \frac{P(x)}{P(\neg x)}$$

3. The Result: This allows the robot to update its map by comparing how likely a measurement is if a wall exists ($P(z|x)$) versus if it doesn't ($P(z|\neg x)$).

 Naren Suri

Occupancy Grid Mapping Algorithm

Occupancy Grid Mapping serves as a foundational framework in robotic perception, enabling a mobile agent to represent an unstructured...

Mar 20  1



See more recommendations